File Uploads ด้วย Node.js + Express

Sep 30, 2026 · @Someone

ภาพรวม

เอกสารนี้สร้าง REST API สำหรับอัปโหลด ดาวน์โหลด และลบไฟล์ด้วย Express + Multer ส่วนการทำ API document ด้วย Swagger แยกไว้ในแท็บ Swagger CRUD API Docs

เบราว์เซอร์หรือแอปส่งไฟล์มาในรูปแบบ `multipart/form-data` ซึ่งแบ่ง body เป็นหลายส่วน (part) คั่นด้วย boundary แต่ละ part มีชื่อฟิลด์ ชื่อไฟล์ และ Content-Type ของตัวเอง Express ไม่อ่าน body แบบนี้เอง จึงต้องใช้ middleware อย่าง Multer มาแยก part และบันทึกไฟล์

ลำดับการทำงานของหนึ่งคำขออัปโหลด:

1. Client ส่ง `POST /api/files` แบบ `multipart/form-data` พร้อมฟิลด์ `file`
2. Multer อ่าน stream แล้วเรียก `fileFilter` เพื่อตรวจชนิดไฟล์
3. Multer ตรวจ `limits` (ขนาดไฟล์ จำนวนไฟล์) ระหว่างรับข้อมูล
4. บันทึกไฟล์ลงดิสก์ (หรือเก็บในหน่วยความจำ) แล้วใส่ข้อมูลไว้ที่ `req.file`
5. Route handler ตอบกลับเป็น JSON ที่มี id, ชื่อ, ขนาด และ URL ของไฟล์
6. ถ้ามีข้อผิดพลาด error middleware แปลงเป็น HTTP status ที่เหมาะสม (400, 413, 415)

Endpoint ที่จะสร้าง:

Method	Path	หน้าที่
POST	`/api/files`	อัปโหลดไฟล์เดียว
POST	`/api/files/multiple`	อัปโหลดหลายไฟล์ (สูงสุด 5)
GET	`/api/files`	รายการไฟล์ทั้งหมด
GET	`/api/files/{filename}`	ดาวน์โหลดไฟล์
DELETE	`/api/files/{filename}`	ลบไฟล์

ตั้งค่าโปรเจกต์

ใช้ Node.js 18 ขึ้นไป สร้างโปรเจกต์และติดตั้งแพ็กเกจดังนี้:

```
mkdir file-upload-api && cd file-upload-api
npm init -y
npm install express multer file-type@16
npm install -D nodemon
```

แพ็กเกจ	หน้าที่
`express`	เว็บเฟรมเวิร์ก
`multer`	อ่าน `multipart/form-data` และบันทึกไฟล์
`file-type@16`	ตรวจชนิดไฟล์จริงจาก magic bytes (v16 ใช้ `require` ได้)

โครงสร้างโฟลเดอร์:

```
file-upload-api/
├── src/
|   ├── app.js              # สร้าง Express app
|   ├── server.js           # เริ่ม server
|   ├── middlewares/
|   |   ├── upload.js       # ตั้งค่า Multer
|   |   ├── verifyFileType.js
|   |   └── errorHandler.js
|   └── routes/
|       └── files.js        # route ของไฟล์
├── uploads/                # ที่เก็บไฟล์ (ใส่ใน .gitignore)
└── package.json
```

เพิ่ม script ใน `package.json`:

```
"scripts": {
  "start": "node src/server.js",
  "dev": "nodemon src/server.js"
}
```

`src/app.js`:

```
const express = require('express');
const fileRoutes = require('./routes/files');
const errorHandler = require('./middlewares/errorHandler');

const app = express();

app.use(express.json());
app.use('/api/files', fileRoutes);

app.use((req, res) => res.status(404).json({ error: 'Not found' }));
app.use(errorHandler); // ต้องอยู่หลังสุดเสมอ

module.exports = app;
```

`src/server.js`:

```
const app = require('./app');
const PORT = process.env.PORT || 3000;

app.listen(PORT, () => console.log(`Server: http://localhost:${PORT}`));
```

อัปโหลดไฟล์ด้วย Multer

Multer มี storage engine หลัก 2 แบบ เลือกตามว่าไฟล์จะไปอยู่ที่ไหนต่อ:

Storage	เก็บที่	เหมาะกับ	ข้อควรระวัง
`diskStorage`	โฟลเดอร์บนเซิร์ฟเวอร์	เก็บไฟล์ถาวรบนเครื่องเดียว	scale หลายเครื่องไม่ได้ถ้าไม่มี shared storage
`memoryStorage`	`req.file.buffer` ใน RAM	ส่งต่อไป S3/Cloud Storage, ย่อรูปทันที	ไฟล์ใหญ่หรือคำขอพร้อมกันมากทำ RAM เต็มได้ ต้องตั้ง `limits`

ตั้งค่า diskStorage

```
const multer = require('multer');
const path = require('path');
const crypto = require('crypto');
const fs = require('fs');

const UPLOAD_DIR = path.join(__dirname, '../../uploads');
fs.mkdirSync(UPLOAD_DIR, { recursive: true });

const storage = multer.diskStorage({
  destination: (req, file, cb) => cb(null, UPLOAD_DIR),
  filename: (req, file, cb) => {
    // ไม่ใช้ชื่อเดิมของผู้ใช้ ป้องกันชื่อซ้ำและ path traversal
    const ext = path.extname(file.originalname).toLowerCase();
    cb(null, `${Date.now()}-${crypto.randomUUID()}${ext}`);
  },
});
```

ถ้าต้องการเก็บในหน่วยความจำแทน ใช้ `const storage = multer.memoryStorage();` แล้วอ่านข้อมูลจาก `req.file.buffer`

รูปแบบการรับไฟล์

Method ของ Multer	ใช้เมื่อ	ผลลัพธ์
`upload.single('file')`	ไฟล์เดียวในฟิลด์ `file`	`req.file`
`upload.array('files', 5)`	หลายไฟล์ในฟิลด์เดียว สูงสุด 5	`req.files` (array)
`upload.fields([{ name: 'avatar', maxCount: 1 }, { name: 'gallery', maxCount: 4 }])`	หลายฟิลด์ต่างชื่อ	`req.files.avatar`, `req.files.gallery`
`upload.none()`	form มีแต่ข้อความ ไม่มีไฟล์	`req.body`

ฟิลด์ข้อความอื่นๆ ใน form จะอยู่ที่ `req.body` เสมอ แต่ Multer เติมให้ตามลำดับที่ client ส่งมา จึงควรให้ client ส่งฟิลด์ข้อความก่อนฟิลด์ไฟล์ ถ้า `fileFilter` ต้องใช้ค่าจาก `req.body`

ข้อมูลใน `req.file` ที่ใช้บ่อย: `originalname` (ชื่อเดิม), `filename` (ชื่อที่บันทึก), `mimetype`, `size` (ไบต์), `path` (เฉพาะ diskStorage), `buffer` (เฉพาะ memoryStorage)

การตรวจสอบไฟล์และความปลอดภัย

อย่าเชื่อข้อมูลจาก client ทั้ง `mimetype` และนามสกุลไฟล์ปลอมได้ง่าย จึงควรตรวจ 3 ชั้น: จำกัดขนาด กรองชนิดไฟล์ตอนรับ และตรวจ magic bytes หลังบันทึก

`src/middlewares/upload.js` (ฉบับสมบูรณ์)

```
const multer = require('multer');
const path = require('path');
const crypto = require('crypto');
const fs = require('fs');

const UPLOAD_DIR = path.join(__dirname, '../../uploads');
fs.mkdirSync(UPLOAD_DIR, { recursive: true });

const ALLOWED = {
  'image/jpeg': ['.jpg', '.jpeg'],
  'image/png': ['.png'],
  'image/webp': ['.webp'],
  'application/pdf': ['.pdf'],
};
const MAX_SIZE = 5 * 1024 * 1024; // 5 MB

const storage = multer.diskStorage({
  destination: (req, file, cb) => cb(null, UPLOAD_DIR),
  filename: (req, file, cb) => {
    const ext = path.extname(file.originalname).toLowerCase();
    cb(null, `${Date.now()}-${crypto.randomUUID()}${ext}`);
  },
});

function fileFilter(req, file, cb) {
  const ext = path.extname(file.originalname).toLowerCase();
  const allowedExts = ALLOWED[file.mimetype];

  if (!allowedExts || !allowedExts.includes(ext)) {
    const err = new Error(`ไม่รองรับไฟล์ชนิด ${file.mimetype} (${ext})`);
    err.status = 415;
    return cb(err);
  }
  cb(null, true);
}

const upload = multer({
  storage,
  fileFilter,
```

```
  limits: {
    fileSize: MAX_SIZE, // ขนาดต่อไฟล์
    files: 5,           // จำนวนไฟล์ต่อคำขอ
    fields: 10,         // จำนวนฟิลด์ข้อความ
  },
});

module.exports = { upload, UPLOAD_DIR, ALLOWED };
```

ตรวจ magic bytes หลังบันทึก

ไฟล์ `.exe` ที่เปลี่ยนชื่อเป็น `.png` จะผ่าน `fileFilter` ได้ เพราะ client ส่ง `mimetype` มาเอง middleware นี้อ่านไบต์แรกของไฟล์จริงแล้วลบทิ้งถ้าไม่ตรง:

```
// src/middlewares/verifyFileType.js
const fs = require('fs/promises');
const FileType = require('file-type'); // v16
const { ALLOWED } = require('./upload');

module.exports = async function verifyFileType(req, res, next) {
  const files = req.file ? [req.file] : req.files || [];
  try {
    for (const f of files) {
      const detected = await FileType.fromFile(f.path);
      if (!detected || !ALLOWED[detected.mime]) {
        await Promise.all(files.map((x) => fs.unlink(x.path).catch(() =>
{})));
        return res.status(415).json({ error: `เนื้อหาไฟล์ ${f.originalname} ไม่ตรง
กับชนิดที่อนุญาต` });
      }
    }
    next();
  } catch (err) {
    next(err);
  }
};
```

Checklist ความปลอดภัย

- ตั้งชื่อไฟล์ใหม่เสมอ ห้ามใช้ `originalname` เป็น path เพราะอาจมี `../` ทำ path traversal
- เก็บไฟล์นอกโฟลเดอร์ที่ serve เป็น static และอย่าให้โฟลเดอร์นั้นรันสคริปต์ได้
- ใช้ allowlist ของชนิดไฟล์ ไม่ใช้ blocklist
- ระวัง SVG และ HTML เพราะฝัง JavaScript ได้ (XSS) ถ้าต้องรับ ให้ส่งกลับพร้อม `Content-Disposition: attachment`

- ตั้ง `limits` ทุกครั้ง โดยเฉพาะกับ `memoryStorage`
- ใส่ rate limit และ authentication ให้ endpoint อัปโหลด
- ระบบที่รับไฟล์จากคนทั่วไป ควรสแกนไวรัส เช่น ClamAV ก่อนเปิดให้ดาวน์โหลด

จัดการ Error และ Route ทั้งหมด

Multer ส่ง error ออกมาเป็น `multer.MulterError` ที่มี `code` บอกสาเหตุ error middleware ตัวเดียวแปลงทุกกรณีเป็น JSON และ status ที่ถูกต้อง

`err.code`	สาเหตุ	Status
`LIMIT_FILE_SIZE`	ไฟล์ใหญ่เกิน `fileSize`	413
`LIMIT_FILE_COUNT`	จำนวนไฟล์เกิน `files`	400
`LIMIT_UNEXPECTED_FILE`	ชื่อฟิลด์ไม่ตรง หรือเกิน `maxCount`	400
`LIMIT_FIELD_COUNT`	ฟิลด์ข้อความเกิน `fields`	400

`src/middlewares/errorHandler.js`

```
const multer = require('multer');

const MESSAGES = {
  LIMIT_FILE_SIZE: 'ไฟล์มีขนาดเกิน 5 MB',
  LIMIT_FILE_COUNT: 'อัปโหลดได้สูงสุด 5 ไฟล์ต่อครั้ง',
  LIMIT_UNEXPECTED_FILE: 'ชื่อฟิลด์ไฟล์ไม่ถูกต้อง หรือจำนวนไฟล์เกินกำหนด',
};

module.exports = (err, req, res, next) => {
  if (err instanceof multer.MulterError) {
    const status = err.code === 'LIMIT_FILE_SIZE' ? 413 : 400;
    return res.status(status).json({ error: MESSAGES[err.code] ||
err.message, code: err.code });
  }

  const status = err.status || 500;
  if (status === 500) console.error(err);
  res.status(status).json({ error: status === 500 ? 'Internal server error' :
err.message });
};
```

`src/routes/files.js`

ใช้ `path.basename()` กับพารามิเตอร์ `filename` เสมอ เพื่อตัด `../` ออกก่อนเอาไปต่อ path

```
const express = require('express');
const path = require('path');
const fs = require('fs/promises');
const { upload, UPLOAD_DIR } = require('../middlewares/upload');
const verifyFileType = require('../middlewares/verifyFileType');

const router = express.Router();

const toDto = (req, f) => ({
  filename: f.filename,
  originalName: f.originalname,
  mimetype: f.mimetype,
  size: f.size,
  url: `${req.protocol}://${req.get('host')}/api/files/${f.filename}`,
});

const safePath = (name) => path.join(UPLOAD_DIR, path.basename(name));

// อัปโหลดไฟล์เดียว
router.post('/', upload.single('file'), verifyFileType, (req, res) => {
  if (!req.file) return res.status(400).json({ error: 'กรุณาแนบไฟล์ในฟิลด์ file'
});
  res.status(201).json(toDto(req, req.file));
});

// อัปโหลดหลายไฟล์
router.post('/multiple', upload.array('files', 5), verifyFileType, (req, res)
=> {
  if (!req.files?.length) return res.status(400).json({ error: 'กรุณาแนบไฟล์ใน
ฟิลด์ files' });
  res.status(201).json({ count: req.files.length, files: req.files.map((f) =>
toDto(req, f)) });
});

// รายการไฟล์
router.get('/', async (req, res, next) => {
  try {
    const names = await fs.readdir(UPLOAD_DIR);
    const files = await Promise.all(
      names.map(async (name) => {
        const stat = await fs.stat(safePath(name));
        return { filename: name, size: stat.size, uploadedAt: stat.birthtime
};
```

```
      })
    );
    res.json(files);
  } catch (err) {
    next(err);
  }
});

// ดาวน์โหลด
router.get('/:filename', (req, res, next) => {
  res.download(safePath(req.params.filename), (err) => {
    if (err && !res.headersSent) {
      err.code === 'ENOENT' ? res.status(404).json({ error: 'ไม่พบไฟล์' }) :
next(err);
    }
  });
});

// ลบ
router.delete('/:filename', async (req, res, next) => {
  try {
    await fs.unlink(safePath(req.params.filename));
    res.status(204).end();
  } catch (err) {
    err.code === 'ENOENT' ? res.status(404).json({ error: 'ไม่พบไฟล์' }) :
next(err);
  }
});

module.exports = router;
```

ทดสอบและ Best Practices สำหรับ Production

ทดสอบด้วย curl

```
# ไฟล์เดียว (-F ทำให้ curl ส่งแบบ multipart/form-data)
curl -F "file=@./avatar.png" http://localhost:3000/api/files

# หลายไฟล์
curl -F "files=@./a.png" -F "files=@./b.pdf"
http://localhost:3000/api/files/multiple

# ดาวน์โหลด (-OJ ใช้ชื่อไฟล์จาก Content-Disposition)
curl -OJ http://localhost:3000/api/files/1727683200000-3f2a9c1e.png

# ลบ
curl -X DELETE -i http://localhost:3000/api/files/1727683200000-3f2a9c1e.png
```

ฝั่ง Frontend

ใช้ `FormData` และห้ามตั้ง header `Content-Type` เอง เพราะเบราว์เซอร์ต้องใส่ `boundary` ให้

```
const form = new FormData();
form.append('description', 'รูปโปรไฟล์'); // ฟิลด์ข้อความก่อนไฟล์
form.append('file', fileInput.files[0]);

const res = await fetch('/api/files', { method: 'POST', body: form });
console.log(await res.json());
```

Best Practices

- **ใช้ Object Storage**: ระบบที่รันหลาย instance ควรเก็บไฟล์ใน S3, Google Cloud Storage หรือ Cloudflare R2 แทนดิสก์ในเครื่อง เช่นใช้ `multer-s3` หรือ `memoryStorage` แล้วอัปโหลดต่อ
- **Presigned URL**: ไฟล์ขนาดใหญ่ (วิดีโอ) ให้ client อัปโหลดตรงไป Object Storage ด้วย presigned URL เพื่อไม่ให้ไฟล์ผ่าน Node.js server
- **เก็บ metadata ในฐานข้อมูล**: บันทึก id, ชื่อเดิม, ขนาด, mimetype, เจ้าของ และ key ใน storage แล้วตรวจสิทธิ์ก่อนดาวน์โหลดหรือลบ
- **ตั้ง limit ที่ reverse proxy ด้วย**: เช่น `client_max_body_size 10m;` ใน Nginx เพื่อตัดคำขอใหญ่ก่อนถึงแอป
- **ประมวลผลรูปแบบ background**: ย่อรูปหรือสร้าง thumbnail ด้วย `sharp` ผ่าน job queue เช่น BullMQ เพื่อไม่ให้ response ช้า
- **ลบไฟล์กำพร้า**: ถ้าบันทึกฐานข้อมูลล้มเหลวหลังไฟล์ลงดิสก์แล้ว ให้ลบไฟล์นั้นทิ้ง และมี cron job เก็บกวาดไฟล์ที่ไม่มี record

Swagger: API Document สำหรับ CRUD ด้วย Node.js + Express

Sep 30, 2026 · @Someone

ภาพรวม

แท็บนี้สร้าง CRUD API ของสินค้า (Products) ด้วย Express แล้วทำเอกสาร API แบบโต้ตอบได้ที่ `/api-docs` ด้วย Swagger ตามมาตรฐาน OpenAPI 3.0

OpenAPI คือมาตรฐานสำหรับอธิบาย REST API ในรูป YAML หรือ JSON ส่วน Swagger คือชุดเครื่องมือที่อ่าน spec นั้น โดย Swagger UI แสดงเป็นหน้าเว็บที่กดทดลองเรียก API ได้ทันที ในโปรเจกต์ Express นิยมใช้ 2 แพ็กเกจคู่กัน:

แพ็กเกจ	หน้าที่
`swagger-jsdoc`	อ่านคอมเมนต์ `@openapi` เหนือ route แล้วรวมเป็น OpenAPI spec
`swagger-ui-express`	เอา spec มาแสดงเป็นหน้า Swagger UI

Endpoint ที่จะ document:

Method	Path	หน้าที่	Status สำเร็จ
GET	`/api/products`	รายการสินค้า (ค้นหา + แบ่งหน้า)	200
GET	`/api/products/{id}`	ดูสินค้าหนึ่งรายการ	200
POST	`/api/products`	สร้างสินค้า	201
PUT	`/api/products/{id}`	แก้ไขทั้งรายการ	200
PATCH	`/api/products/{id}`	แก้ไขบางฟิลด์	200
DELETE	`/api/products/{id}`	ลบสินค้า	204

ตั้งค่าโปรเจกต์และ Swagger

```
mkdir crud-api && cd crud-api
npm init -y
npm install express swagger-jsdoc swagger-ui-express
npm install -D nodemon
```

```
crud-api/
├── src/
|   ├── app.js
|   ├── server.js
|   ├── config/
|   |   └── swagger.js      # definition + schemas
|   └── routes/
|       └── products.js     # CRUD + คอมเมนต์ @openapi
└── package.json
```

`src/config/swagger.js`

เก็บ schema ที่ใช้ซ้ำไว้ใน `components` แล้วอ้างด้วย `$ref` แยก `Product` (ข้อมูลที่ตอบกลับ มี `id`, `createdAt`) ออกจาก `ProductInput` (ข้อมูลที่ client ส่งมา) เพราะ client ไม่ควรกำหนด `id` เอง

```
const path = require('path');
const swaggerJsdoc = require('swagger-jsdoc');

module.exports = swaggerJsdoc({
  definition: {
    openapi: '3.0.3',
    info: {
      title: 'Products CRUD API',
      version: '1.0.0',
      description: 'ตัวอย่าง CRUD API พร้อมเอกสารด้วย Swagger',
    },
    servers: [{ url: 'http://localhost:3000', description: 'Local' }],
    tags: [{ name: 'Products', description: 'จัดการสินค้า' }],
    components: {
      schemas: {
        ProductInput: {
          type: 'object',
          required: ['name', 'price'],
          properties: {
            name: { type: 'string', minLength: 1, example: 'คีย์บอร์ดไร้สาย' },
            price: { type: 'number', minimum: 0, example: 1290 },
            stock: { type: 'integer', minimum: 0, default: 0, example: 25 },
            category: { type: 'string', enum: ['electronics', 'books',
```

```
'fashion'], example: 'electronics' },
          },
        },
        Product: {
          allOf: [
            {
              type: 'object',
              properties: {
                id: { type: 'integer', readOnly: true, example: 1 },
                createdAt: { type: 'string', format: 'date-time', readOnly:
true },
              },
            },
            { $ref: '#/components/schemas/ProductInput' },
          ],
        },
        Error: {
          type: 'object',
          properties: { error: { type: 'string', example: 'ไม่พบสินค้า' } },
        },
      },
      parameters: {
        ProductId: {
          in: 'path',
          name: 'id',
          required: true,
          schema: { type: 'integer', minimum: 1 },
          description: 'รหัสสินค้า',
        },
      },
      responses: {
        NotFound: {
          description: 'ไม่พบสินค้า',
          content: { 'application/json': { schema: { $ref:
'#/components/schemas/Error' } } },
        },
        BadRequest: {
          description: 'ข้อมูลไม่ถูกต้อง',
          content: { 'application/json': { schema: { $ref:
'#/components/schemas/Error' } } },
        },
      },
    },
  },
  apis: [path.join(__dirname, '../routes/*.js')],
});
```

`src/app.js` และ `src/server.js`

```
// src/app.js
const express = require('express');
const swaggerUi = require('swagger-ui-express');
const swaggerSpec = require('./config/swagger');
const productRoutes = require('./routes/products');

const app = express();
app.use(express.json());

app.use('/api-docs', swaggerUi.serve, swaggerUi.setup(swaggerSpec));
app.get('/api-docs.json', (req, res) => res.json(swaggerSpec)); // spec ดิบ

app.use('/api/products', productRoutes);
app.use((req, res) => res.status(404).json({ error: 'Not found' }));

module.exports = app;

// src/server.js
const app = require('./app');
const PORT = process.env.PORT || 3000;
app.listen(PORT, () => console.log(`Swagger: http://localhost:${PORT}/api-
docs`));
```

สร้าง CRUD Routes

ตัวอย่างนี้เก็บข้อมูลใน array เพื่อให้โฟกัสที่ Swagger ได้ งานจริงเปลี่ยนเป็นฐานข้อมูล (เช่น PostgreSQL ผ่าน Prisma หรือ MongoDB ผ่าน Mongoose) โดยไม่ต้องแก้ส่วนเอกสาร

`src/routes/products.js`

```
const express = require('express');
const router = express.Router();

let products = [
  { id: 1, name: 'คีย์บอร์ดไร้สาย', price: 1290, stock: 25, category:
'electronics', createdAt: new Date().toISOString() },
  { id: 2, name: 'หนังสือ Node.js', price: 450, stock: 10, category: 'books',
createdAt: new Date().toISOString() },
];
let nextId = 3;

const CATEGORIES = ['electronics', 'books', 'fashion'];
```

```
function validate(body, partial = false) {
  const { name, price, stock, category } = body;
  if (!partial || name !== undefined) {
    if (typeof name !== 'string' || !name.trim()) return 'name ต้องเป็นข้อความและ
ไม่ว่าง';
  }
  if (!partial || price !== undefined) {
    if (typeof price !== 'number' || price < 0) return 'price ต้องเป็นตัวเลขที่ไม่
ติดลบ';
  }
  if (stock !== undefined && (!Number.isInteger(stock) || stock < 0)) return
'stock ต้องเป็นจำนวนเต็มที่ไม่ติดลบ';
  if (category !== undefined && !CATEGORIES.includes(category)) return
`category ต้องเป็น ${CATEGORIES.join(', ')}`;
  return null;
}

const findProduct = (req, res, next) => {
  const product = products.find((p) => p.id === Number(req.params.id));
  if (!product) return res.status(404).json({ error: 'ไม่พบสินค้า' });
  req.product = product;
  next();
};

// READ ทั้งหมด: ?q=คำค้น&category=books&page=1&limit=10
router.get('/', (req, res) => {
  const { q, category } = req.query;
  const page = Math.max(1, parseInt(req.query.page) || 1);
  const limit = Math.min(100, parseInt(req.query.limit) || 10);

  let result = products;
  if (q) result = result.filter((p) =>
p.name.toLowerCase().includes(q.toLowerCase()));
  if (category) result = result.filter((p) => p.category === category);

  const start = (page - 1) * limit;
  res.json({ data: result.slice(start, start + limit), page, limit, total:
result.length });
});

// READ หนึ่งรายการ
router.get('/:id', findProduct, (req, res) => res.json(req.product));

// CREATE
router.post('/', (req, res) => {
  const error = validate(req.body);
```

```
  if (error) return res.status(400).json({ error });

  const { name, price, stock = 0, category } = req.body;
  const product = { id: nextId++, name, price, stock, category, createdAt:
new Date().toISOString() };
  products.push(product);
  res.status(201).location(`/api/products/${product.id}`).json(product);
});

// UPDATE ทั้งรายการ
router.put('/:id', findProduct, (req, res) => {
  const error = validate(req.body);
  if (error) return res.status(400).json({ error });

  const { name, price, stock = 0, category } = req.body;
  Object.assign(req.product, { name, price, stock, category });
  res.json(req.product);
});

// UPDATE บางฟิลด์
router.patch('/:id', findProduct, (req, res) => {
  const error = validate(req.body, true);
  if (error) return res.status(400).json({ error });

  const allowed = ['name', 'price', 'stock', 'category'];
  for (const key of allowed) if (req.body[key] !== undefined)
req.product[key] = req.body[key];
  res.json(req.product);
});

// DELETE
router.delete('/:id', findProduct, (req, res) => {
  products = products.filter((p) => p.id !== req.product.id);
  res.status(204).end();
});

module.exports = router;
```

ความต่างของ PUT กับ PATCH: PUT ส่งข้อมูลครบทุกฟิลด์และแทนที่ทั้งหมด ฟิลด์ที่ไม่ส่งจะกลับเป็นค่าเริ่มต้น ส่วน PATCH ส่งเฉพาะฟิลด์ที่ต้องการเปลี่ยน

เขียน OpenAPI Annotation ให้แต่ละ Endpoint

วางคอมเมนต์ `@openapi` ไว้เหนือ route ใน `src/routes/products.js` หนึ่งคอมเมนต์รวมหลาย method ของ path เดียวกันได้ ใช้ `$ref` ชี้ไปที่ schema, parameter และ response ใน `components` เพื่อไม่ต้องเขียนซ้ำ

`/api/products` : GET (list) และ POST (create)

```
/**
 * @openapi
 * /api/products:
 *   get:
 *     tags: [Products]
 *     summary: รายการสินค้า
 *     parameters:
 *       - in: query
 *         name: q
 *         schema: { type: string }
 *         description: ค้นหาจากชื่อสินค้า
 *       - in: query
 *         name: category
 *         schema: { type: string, enum: [electronics, books, fashion] }
 *       - in: query
 *         name: page
 *         schema: { type: integer, minimum: 1, default: 1 }
 *       - in: query
 *         name: limit
 *         schema: { type: integer, minimum: 1, maximum: 100, default: 10 }
 *     responses:
 *       200:
 *         description: สำเร็จ
 *         content:
 *           application/json:
 *             schema:
 *               type: object
 *               properties:
 *                 data:
 *                   type: array
 *                   items: { $ref: '#/components/schemas/Product' }
 *                 page: { type: integer, example: 1 }
 *                 limit: { type: integer, example: 10 }
 *                 total: { type: integer, example: 2 }
 *   post:
 *     tags: [Products]
 *     summary: สร้างสินค้าใหม่
 *     requestBody:
```

```
 *       required: true
 *       content:
 *         application/json:
 *           schema: { $ref: '#/components/schemas/ProductInput' }
 *     responses:
 *       201:
 *         description: สร้างสำเร็จ
 *         headers:
 *           Location:
 *             schema: { type: string }
 *             description: URL ของสินค้าที่สร้าง
 *         content:
 *           application/json:
 *             schema: { $ref: '#/components/schemas/Product' }
 *       400:
 *         $ref: '#/components/responses/BadRequest'
 */
router.get('/', /* ... */);
router.post('/', /* ... */);
```

`/api/products/{id}` : GET, PUT, PATCH, DELETE

`parameters` ที่วางระดับ path ใช้ร่วมกันทุก method ส่วน PATCH ใช้ schema ที่ไม่มี `required` เพราะส่งแค่บางฟิลด์ได้

```
/**
 * @openapi
 * /api/products/{id}:
 *   parameters:
 *     - $ref: '#/components/parameters/ProductId'
 *   get:
 *     tags: [Products]
 *     summary: ดูสินค้าตาม id
 *     responses:
 *       200:
 *         description: สำเร็จ
 *         content:
 *           application/json:
 *             schema: { $ref: '#/components/schemas/Product' }
 *       404:
 *         $ref: '#/components/responses/NotFound'
 *   put:
 *     tags: [Products]
 *     summary: แก้ไขสินค้าทั้งรายการ
 *     requestBody:
 *       required: true
```

```
 *       content:
 *         application/json:
 *           schema: { $ref: '#/components/schemas/ProductInput' }
 *     responses:
 *       200:
 *         description: แก้ไขสำเร็จ
 *         content:
 *           application/json:
 *             schema: { $ref: '#/components/schemas/Product' }
 *       400:
 *         $ref: '#/components/responses/BadRequest'
 *       404:
 *         $ref: '#/components/responses/NotFound'
 *   patch:
 *     tags: [Products]
 *     summary: แก้ไขสินค้าบางฟิลด์
 *     requestBody:
 *       required: true
 *       content:
 *         application/json:
 *           schema:
 *             type: object
 *             minProperties: 1
 *             properties:
 *               name: { type: string }
 *               price: { type: number, minimum: 0 }
 *               stock: { type: integer, minimum: 0 }
 *               category: { type: string, enum: [electronics, books,
fashion] }
 *           example: { price: 990, stock: 40 }
 *     responses:
 *       200:
 *         description: แก้ไขสำเร็จ
 *         content:
 *           application/json:
 *             schema: { $ref: '#/components/schemas/Product' }
 *       400:
 *         $ref: '#/components/responses/BadRequest'
 *       404:
 *         $ref: '#/components/responses/NotFound'
 *   delete:
 *     tags: [Products]
 *     summary: ลบสินค้า
 *     responses:
 *       204:
 *         description: ลบสำเร็จ (ไม่มี body)
```

```
 *       404:
 *         $ref: '#/components/responses/NotFound'
 */
router.get('/:id', /* ... */);
```

สังเกตว่า Express ใช้ `:id` แต่ OpenAPI ใช้ `{id}` ถ้าเขียนผิดรูปแบบ Swagger UI จะไม่แทนค่า id ให้ตอนกด Try it out

ส่วนของ annotation	ใช้ทำอะไร
`tags`	จัดกลุ่ม endpoint ในหน้า Swagger UI
`summary` / `description`	คำอธิบายสั้น / ยาว
`parameters`	ค่าจาก `path`, `query`, `header`, `cookie`
`requestBody`	body ที่ client ส่งมา พร้อม content type
`responses`	ทุก status ที่ endpoint อาจตอบ พร้อม schema
`$ref`	อ้างถึงของที่นิยามไว้ใน `components`

ทดสอบและแนวทางสำหรับ Production

ทดสอบผ่าน Swagger UI

1. รัน `npx nodemon src/server.js` แล้วเปิด `http://localhost:3000/api-docs`
2. เปิด `POST /api/products` → **Try it out** แก้ JSON ตัวอย่าง แล้วกด **Execute**
3. ดู response `201` และคำสั่ง curl ที่ Swagger สร้างให้
4. ลอง `GET /api/products/{id}` ใส่ id ที่ได้ แล้วลอง `PATCH`, `DELETE` ตามลำดับ
5. ส่ง `price: -1` หรือ id ที่ไม่มีอยู่ เพื่อตรวจว่า `400` และ `404` ตรงกับเอกสาร

เพิ่ม Authentication ในเอกสาร

ประกาศ `securitySchemes` ใน `components` แล้ว Swagger UI จะมีปุ่ม **Authorize** ให้ใส่ token ครั้งเดียวใช้ได้ทุก endpoint:

```
// ใน definition ของ src/config/swagger.js
components: {
  securitySchemes: {
    bearerAuth: { type: 'http', scheme: 'bearer', bearerFormat: 'JWT' },
  },
  // schemas, parameters, responses ...
},
security: [{ bearerAuth: [] }], // บังคับทุก endpoint
```

endpoint ที่เปิดสาธารณะ เช่น `GET /api/products` ใส่ `security: []` ใน annotation ของ method นั้นเพื่อยกเว้น และเพิ่ม response `401` ให้ endpoint ที่ต้องใช้ token

ทางเลือก: เขียน spec เป็นไฟล์ YAML

ถ้า API ใหญ่ คอมเมนต์ JSDoc จะรกไฟล์ route วิธีที่นิยมคือเขียน `openapi.yaml` แยก แล้วโหลดด้วย `yamljs` หรือ `js-yaml`:

```
const fs = require('fs');
const yaml = require('js-yaml');
const swaggerSpec = yaml.load(fs.readFileSync('./openapi.yaml', 'utf8'));
app.use('/api-docs', swaggerUi.serve, swaggerUi.setup(swaggerSpec));
```

วิธี	ข้อดี	ข้อเสีย
คอมเมนต์ `@openapi` (swagger-jsdoc)	เอกสารอยู่ติดโค้ด แก้พร้อมกัน	YAML ในคอมเมนต์ไม่มี syntax highlight เยื้องผิดง่าย
ไฟล์ `openapi.yaml` แยก	อ่านง่าย ใช้ Swagger Editor ตรวจได้ ทำ design-first ได้	ต้องคอยอัปเดตให้ตรงกับโค้ดเอง

แนวทางอื่นๆ

- **ตรวจ YAML**: เยื้องด้วยช่องว่างเท่านั้น ห้ามใช้ Tab ถ้าเยื้องผิด endpoint จะหายจากหน้าเอกสารโดยไม่มี error ให้เปิด `/api-docs.json` ตรวจทุกครั้ง
- **ใช้ spec ตรวจ request จริง**: `express-openapi-validator` อ่าน spec เดียวกันแล้ว validate body, query และ path ให้อัตโนมัติ ทำให้เอกสารกับพฤติกรรมจริงไม่เพี้ยนกัน
- **ป้องกันหน้าเอกสาร**: ใน production ใส่ Basic Auth ให้ `/api-docs` หรือเปิดเฉพาะใน staging
- **ใช้ spec ต่อ**: import `/api-docs.json` เข้า Postman หรือสร้าง client SDK ด้วย OpenAPI Generator
- **ใส่ `example` ให้ครบ**: Swagger UI เอาไปเติมฟอร์ม Try it out ให้ ช่วยให้ทีม frontend ทดลองได้เร็ว
- **กำหนดเวอร์ชัน**: เปลี่ยน `info.version` เมื่อ API เปลี่ยน และใช้ path อย่าง `/api/v1` เมื่อมี breaking change